

Data Wrangling 🤠

Math for Naive Bayes

Overview

- Our model will look at some text and assign it a label:
 - "this movie was boring and dumb" → negative
- Some words are strong evidence for a label while some words are noise
- How do we take these words and turn them into prediction scores or numbers?
 - We will discuss all the math you need to do for the project

Your Training Data

Every row is one labeled example with at least one `label` and some `content`:

<code>label</code>	<code>content</code>
positive	wonderful acting and story
positive	wonderful movie loved it
negative	boring acting and dull
negative	terrible movie hated it

- This is a tiny version of your `train.csv`

Training Steps

Before it can predict anything, your classifier reads the training data **once** and count four things:

Symbol	Meaning
N	total number of training examples
N_C	number of examples with label C
N_w	number of examples that contain word w (any label)
$N_{(C,w)}$	number of examples with label C that contain word w

- Everything you compute later comes from these four counts

Counting Examples: N and N_C

From our 4-row table:

- **N** = 4 (total examples)
- **N_positive** = 2
- **N_negative** = 2

Counting Words: N_w

- First, remember **bag of words**: for each example, we only care about *which* words appear, not how many times
- "wonderful acting and story" → the set
{wonderful, acting, and, story}

Counting how many examples (out of all 4, regardless of label) contain each word:

word	N_w
wonderful	2
acting	2

Counting Words *Per Label*: $N_{\text{C,w}}$

Now we count and split by label. Keep track of how many **positive** examples contain each word, and how many **negative** examples contain each word:

Positive words:

word	$N_{\text{positive, w}}$
wonderful	2
acting	1
movie	1
h	1

Counting Words *Per Label*: $N_{\text{C},w}$

Negative words:

word	$N_{\text{negative}, w}$
wonderful	0
acting	1
movie	1
and	1
story	0
loved	0

N_w compared to $N(C, w)$

word	$N(\text{positive}, w)$	$N(\text{negative}, w)$
wonderful	2	0
acting	1	1
movie	1	1
and	1	1
story	1	0
loved	1	0
it	1	1
boring	0	1

Training is Done!

- Everything we compute regarding prediction will come from these 4 counts: $\mathbf{N}$, $\mathbf{N}_\mathbf{C}$, $\mathbf{N}_\mathbf{w}$, $\mathbf{N}_\mathbf{(C,w)}$
- Now, as we see new text examples, we ask:
 1. How common is each label, on its own?
 2. How well do the the words in this fit review (i.e. text) each label?

Step 1: How common is each label?

- **log-prior:** Before even reading the words in a new review, consider how likely each label just based on how often it shows up in training data.

$$\text{log-prior}(C) = \ln(N_C \div N)$$

For our dataset:

- $\text{log-prior}(\text{positive}) = \ln(2 \div 4) = \ln(0.5) \approx \mathbf{-0.693}$
- $\text{log-prior}(\text{negative}) = \ln(2 \div 4) = \ln(0.5) \approx \mathbf{-0.693}$

A Note on Logarithms

- Why not just multiply plain probabilities? Because multiplying many small fractions together produces **extremely** tiny numbers!
- Logs are also **monotonic**: bigger input always gives a bigger output so comparing log-scores directly is exactly as valid as comparing the original probabilities
-  **You cannot take the log of zero!**

Step 2: How well does a word fit a label?

- ***log-likelihood**: If we are considering label C, how much does the presence of a single word w in the review support that?

$$\text{log-likelihood}(w, C) = \ln(N_{(C,w)} \div N_C)$$

For our dataset considering label positive:

- Remember, $N_{\text{positive}} = 2$
- **wonderful**: $N_{(\text{positive}, \text{wonderful})} = 2 \rightarrow \ln(2 \div 2) = \ln(1) = 0$

Step 2, cont'd: What is a word doesn't fit perfectly?

What if a word never showed up with this label at all? **Three cases, checked in order:**

Case	Condition	Formula
1	word appeared with this label at least once	$\ln(N_{(C,w)} \div N_C)$
2	word appeared in training, but never with this label	$\ln(N_w \div N)$
3	word never appeared in training at all	$\ln(1 \div N)$

Example: Choosing a Fallback Case

`wonderful` never appeared in any **negative** example:
 $N_{\text{negative}, \text{wonderful}} = 0$.

- Case 1 doesn't apply (0 examples)
- Check case 2: did `wonderful` appear in training at all? Yes, we see that $N_{\text{wonderful}} = 2$. **So we can use case 2.**

$\text{log-likelihood}(\text{wonderful}, \text{negative}) = \ln(N_w \div N) = \ln(2 \div 4) = \ln(0.5) \approx -0.693$

- Compare to `wonderful` under **positive**, which scored exactly **0**! Do you think the word `wonderful` supports the label `negative` or

Concept Check

Using the $N_{(C,w)}$ table from a few slides ago, which case (1, 2, or 3) applies to each, and why?

- **boring** under label **positive**
- **hated** under label **negative**
- **amazing** under label **positive** (a word that never appeared anywhere in training)

Step 3: Scoring a Whole Piece of Text

To score an entire piece of text against a label, combine the log-prior with the log-likelihood of **every unique word** in the text:

$\text{score}(\text{text}, C) = \text{log-prior}(C) + \text{sum of log-likelihood}(w, C) \text{ for each unique word } w \text{ in text}$

- Start with how common the label is on its own
- Add one number per unique word, showing how well that word fits this label
- More words that fit well will mean higher (less negative) score for that label

Example: Scoring

"wonderful acting movie" under

positive

New, unlabeled text: **"wonderful acting movie"** → unique words:

{wonderful, acting, movie}

Term	Value
log-prior(positive)	-0.693
log-likelihood(wonderful, positive)	0
log-likelihood(acting, positive)	-0.693

Example: Scoring

"wonderful acting movie" under
positive

Same text, scored against **negative**:

Term	Value
log-prior(negative)	-0.693
log-likelihood(wonderful, negative)	-0.693 (fallback case 2)
log-likelihood(acting, negative)	-0.693
log-likelihood(movie, negative)	-0.693

Step 4: Making a Prediction

- Compute **score(text, C)** for **every possible label**
- Predict whichever label has the **highest** score
- If there is a tie, predict whichever label comes first **alphabetically**

For our example: $\text{score}(\text{positive}) = -2.079 > \text{score}(\text{negative}) = -2.772$

Prediction: positive 

Step 5: Evaluating on Test Data

Once training is done, run this same process on **every row of your test file**:

1. Predict a label for the text
2. Compare the prediction to the row's true label
3. Count how many predictions were correct

accuracy = (number of correct predictions) ÷ (total number of test examples)

Formula Cheat Sheet

Step	Formula
Log-prior	$\ln(N_C \div N)$
Log-likelihood (case 1)	$\ln(N_{(C,w)} \div N_C)$
Log-likelihood (case 2)	$\ln(N_w \div N)$
Log-likelihood (case 3)	$\ln(1 \div N)$
Score	$\text{log-prior}(C) + \text{sum of log-likelihood}(w, C)$
Prediction	label C with the highest score(text, C)
Accuracy	correct predictions $\div$ total predictions

Summary

- Training a Naive Bayes classifier means counting four things once: N , N_C , N_w , $N_{(C,w)}$
- **Log-prior** measures how common a label is on its own; **log-likelihood** measures how well one word fits a label
- Choose a *fallback case* for unseen or rare words
- **Score** = log-prior + the sum of log-likelihoods for every unique word in the text
- **Predict** = whichever label scores highest; **accuracy** = correct ÷ total on the test set